

SCALE × ODYSSEY

Sequence-Based Classification of Astronomical Objects Using Deep Learning

Complete Starter Guide

TechOIITGN Hackathon | Team Submission Reference

This guide provides everything you need to go from zero to a working deep learning pipeline for astronomical image classification. It covers environment setup, dataset preparation, model architecture, training, evaluation, explainability, and deployment.

Table of Contents

1. Project Overview & Architecture Decision
2. Environment Setup
3. Project Folder Structure
4. Dataset Download & Preparation
5. Data Preprocessing Pipeline
6. Model Architectures
7. Training Script
8. Evaluation & Metrics
9. Grad-CAM Explainability
10. Streamlit Web App
11. Evaluation Criteria Checklist
12. Submission Checklist

1. Project Overview & Architecture Decision

Goal

Classify raw astronomical images into 5 categories using a deep learning model trained end-to-end — no manual feature engineering.

Class Label	Examples
Spiral Galaxy	Milky Way-type structures with arms
Elliptical Galaxy	Smooth, featureless oval blobs
Nebula	Gas/dust clouds, often colorful
Star Cluster	Dense groupings of stars
Planetary Object	Planets, moons, rings

Recommended Architecture

Use **EfficientNet-B3** as your primary backbone with transfer learning. It gives the best accuracy/efficiency trade-off for this task on consumer hardware.

Model	Top-1 Accuracy (ImageNet)	Params	Inference Speed
ResNet-50	~76%	25M	Fast
EfficientNet-B3 	~82%	12M	Fast
EfficientNet-B5	~84%	30M	Medium
ViT-B/16	~85%	86M	Slower

■ Start with EfficientNet-B3. If accuracy is below 80%, upgrade to B5 or add a ViT ensemble.

2. Environment Setup

Option A: Local (Conda)

```
conda create -n scale_odyssey python=3.10 -y
conda activate scale_odyssey

# PyTorch with CUDA 11.8
pip install torch torchvision torchaudio --index-url https://download.pytorch.org/whl/cu118

# Core libraries
pip install timm alumentations opencv-python scikit-learn matplotlib seaborn
pip install grad-cam streamlit fastapi uvicorn pillow tqdm pandas numpy
pip install kaggle # for dataset download
```

Option B: Google Colab / Kaggle Notebook

```
!pip install timm alumentations grad-cam streamlit -q

# Check GPU
import torch
print(torch.cuda.is_available()) # Should print True
print(torch.cuda.get_device_name(0))
```

Verify Installation

```
import torch, timm, cv2, alumentations
print("PyTorch:", torch.__version__)
print("TIMM:", timm.__version__)
print("GPU available:", torch.cuda.is_available())
```

3. Project Folder Structure

```
scale_odyssey/
├── data/
│   ├── raw/                # Downloaded datasets
│   ├── processed/          # After preprocessing
│   └── train/
│       ├── spiral_galaxy/
│       ├── elliptical_galaxy/
│       ├── nebula/
│       ├── star_cluster/
│       ├── planetary/
│       ├── val/
│       └── test/
│   ├── download_datasets.py
│   └──
├── src/
│   ├── dataset.py          # PyTorch Dataset class
│   ├── model.py            # Model architecture
│   ├── train.py            # Training loop
│   ├── evaluate.py         # Evaluation & metrics
│   ├── gradcam.py          # Grad-CAM visualization
│   └── utils.py            # Helper functions
│   └──
├── notebooks/
│   ├── 01_EDA.ipynb        # Exploratory Data Analysis
│   ├── 02_Training.ipynb   # Training experiments
│   └── 03_Evaluation.ipynb # Results & visualization
│   └──
├── app/
│   ├── app.py              # Streamlit web app
│   └── inference.py         # Inference wrapper
│   └──
├── checkpoints/            # Saved model weights
├── results/                # Plots, confusion matrices
├── requirements.txt
└── README.md
```

4. Dataset Download & Preparation

Available Datasets

Dataset	Source	Best For
Galaxy Zoo	Zooniverse / Kaggle	Spiral & Elliptical
DeepSky Dataset	Kaggle	Nebulae, Clusters
Hubble Image Archive	NASA/ESA (hubblesite.org)	High-res all classes
Astronomy Classification	Kaggle (preprocessed)	Quick start / baseline

Kaggle API Download

```
# Setup Kaggle credentials first
# Go to kaggle.com > Account > Create API Token
# Place kaggle.json in ~/.kaggle/

import os, zipfile
os.makedirs("data/raw", exist_ok=True)

# Galaxy Zoo dataset
os.system("kaggle datasets download -d jaimetriczk/galaxy-classification -p data/raw/")

# Astronomy Classification Dataset
os.system("kaggle datasets download -d fedesoriano/astronomy-dataset -p data/raw/")

# Unzip all
for f in os.listdir("data/raw/"):
    if f.endswith(".zip"):
        with zipfile.ZipFile(f"data/raw/{f}", 'r') as z:
            z.extractall("data/raw/")
        print(f"Extracted: {f}")
```

Organize Data into Class Folders

```
import os, shutil, random
from pathlib import Path

CLASS_MAP = {
    "spiral": "spiral_galaxy",
    "elliptical": "elliptical_galaxy",
    "nebula": "nebula",
    "star_cluster": "star_cluster",
    "planet": "planetary",
}

SPLIT = {"train": 0.75, "val": 0.15, "test": 0.10}

def organize_dataset(source_dir, output_dir):
    for class_name, folder_name in CLASS_MAP.items():
        images = list(Path(source_dir).rglob(f"*{class_name}/*.jpg"))
        random.shuffle(images)
        n = len(images)
        splits = {
```

```
    "train": images[:int(n*0.75)],
    "val":    images[int(n*0.75):int(n*0.90)],
    "test":   images[int(n*0.90):]
}
for split, imgs in splits.items():
    dest = Path(output_dir) / split / folder_name
    dest.mkdir(parents=True, exist_ok=True)
    for img in imgs:
        shutil.copy(img, dest / img.name)
    print(f"{folder_name}: {n} images split")

organize_dataset("data/raw", "data/processed")
```

5. Data Preprocessing Pipeline

dataset.py — PyTorch Dataset with Augmentation

```
import os
import torch
import numpy as np
from PIL import Image
from torch.utils.data import Dataset, DataLoader
from torchvision import transforms, datasets
import albumentations as A
from albumentations.pytorch import ToTensorV2

CLASSES = ["spiral_galaxy", "elliptical_galaxy", "nebula",
            "star_cluster", "planetary"]

# Strong augmentations for training
train_transform = A.Compose([
    A.RandomRotate90(p=0.5),
    A.HorizontalFlip(p=0.5),
    A.VerticalFlip(p=0.3),
    A.RandomBrightnessContrast(brightness_limit=0.3,
                               contrast_limit=0.3, p=0.5),
    A.GaussNoise(var_limit=(10, 50), p=0.3),
    A.CoarseDropout(max_holes=8, max_height=16,
                    max_width=16, p=0.3),
    A.Resize(224, 224),
    A.Normalize(mean=[0.485, 0.456, 0.406],
                std=[0.229, 0.224, 0.225]),
    ToTensorV2()
])

# Minimal transforms for val/test
val_transform = A.Compose([
    A.Resize(224, 224),
    A.Normalize(mean=[0.485, 0.456, 0.406],
                std=[0.229, 0.224, 0.225]),
    ToTensorV2()
])

class AstroDataset(Dataset):
    def __init__(self, root_dir, split='train'):
        self.transform = train_transform if split=='train' else val_transform
        self.samples = []
        self.labels = []
        for idx, cls in enumerate(CLASSES):
            folder = os.path.join(root_dir, split, cls)
            if not os.path.exists(folder): continue
            for img_name in os.listdir(folder):
                if img_name.lower().endswith(('.jpg', '.png', '.jpeg')):
                    self.samples.append(os.path.join(folder, img_name))
                    self.labels.append(idx)

    def __len__(self):
        return len(self.samples)

    def __getitem__(self, idx):
        img = np.array(Image.open(self.samples[idx]).convert('RGB'))
```



```

        if self.transform:
            img = self.transform(image=img)['image']
        return img, self.labels[idx]

def get_loaders(data_dir, batch_size=32, num_workers=4):
    train_ds = AstroDataset(data_dir, 'train')
    val_ds   = AstroDataset(data_dir, 'val')
    test_ds  = AstroDataset(data_dir, 'test')
    train_loader = DataLoader(train_ds, batch_size=batch_size,
                              shuffle=True, num_workers=num_workers,
                              pin_memory=True)
    val_loader   = DataLoader(val_ds, batch_size=batch_size,
                              shuffle=False, num_workers=num_workers)
    test_loader  = DataLoader(test_ds, batch_size=batch_size,
                              shuffle=False, num_workers=num_workers)
    print(f"Train: {len(train_ds)} | Val: {len(val_ds)} | Test: {len(test_ds)}")
    return train_loader, val_loader, test_loader

```

6. Model Architecture

model.py — EfficientNet-B3 with Custom Head

```
import torch
import torch.nn as nn
import timm

class AstroClassifier(nn.Module):
    def __init__(self, num_classes=5, backbone='efficientnet_b3',
                  pretrained=True, dropout=0.3):
        super().__init__()

        # Load pretrained backbone from TIMM
        self.backbone = timm.create_model(backbone,
                                          pretrained=pretrained,
                                          num_classes=0) # remove head
        in_features = self.backbone.num_features

        # Custom classification head
        self.classifier = nn.Sequential(
            nn.AdaptiveAvgPool2d(1),
            nn.Flatten(),
            nn.BatchNorm1d(in_features),
            nn.Dropout(dropout),
            nn.Linear(in_features, 512),
            nn.ReLU(inplace=True),
            nn.BatchNorm1d(512),
            nn.Dropout(dropout / 2),
            nn.Linear(512, num_classes)
        )

    def forward(self, x):
        features = self.backbone.forward_features(x)
        out = self.classifier(features)
        return out

# Instantiate and verify
if __name__ == '__main__':
    model = AstroClassifier(num_classes=5)
    x = torch.randn(4, 3, 224, 224)
    out = model(x)
    print("Output shape:", out.shape) # (4, 5)
    total_params = sum(p.numel() for p in model.parameters())
    print(f"Total parameters: {total_params:,}")
```

Alternative: Vision Transformer (ViT)

```
# Swap backbone easily using TIMM
model_vit = timm.create_model('vit_base_patch16_224',
                              pretrained=True, num_classes=5)

# Or use a hybrid approach
model_hybrid = timm.create_model('efficientformer_l1',
                                  pretrained=True, num_classes=5)
```

■ Use EfficientNet-B3 first. ViT needs more data and longer training to shine.

7. Training Script

train.py — Full Training Loop with Best Practices

```
import torch
import torch.nn as nn
import torch.optim as optim
from torch.optim.lr_scheduler import CosineAnnealingLR
from tqdm import tqdm
import os

from dataset import get_loaders
from model import AstroClassifier

# Config
CONFIG = {
    'data_dir': 'data/processed',
    'batch_size': 32,
    'num_epochs': 30,
    'lr': 3e-4,
    'weight_decay': 1e-4,
    'num_classes': 5,
    'backbone': 'efficientnet_b3',
    'checkpoint_dir': 'checkpoints',
    'patience': 7, # early stopping
}

DEVICE = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print(f"Using device: {DEVICE}")

def train_one_epoch(model, loader, optimizer, criterion, scaler):
    model.train()
    total_loss, correct, total = 0.0, 0, 0
    for images, labels in tqdm(loader, desc='Training'):
        images, labels = images.to(DEVICE), labels.to(DEVICE)
        optimizer.zero_grad()
        with torch.cuda.amp.autocast(): # Mixed precision
            outputs = model(images)
            loss = criterion(outputs, labels)
        scaler.scale(loss).backward()
        scaler.step(optimizer)
        scaler.update()
        total_loss += loss.item() * images.size(0)
        preds = outputs.argmax(dim=1)
        correct += (preds == labels).sum().item()
        total += images.size(0)
    return total_loss / total, correct / total

def validate(model, loader, criterion):
    model.eval()
    total_loss, correct, total = 0.0, 0, 0
    with torch.no_grad():
        for images, labels in tqdm(loader, desc='Validating'):
            images, labels = images.to(DEVICE), labels.to(DEVICE)
            outputs = model(images)
            loss = criterion(outputs, labels)
            total_loss += loss.item() * images.size(0)
            preds = outputs.argmax(dim=1)
```

```

        correct += (preds == labels).sum().item()
        total += images.size(0)
    return total_loss / total, correct / total

def main():
    os.makedirs(CONFIG['checkpoint_dir'], exist_ok=True)
    train_loader, val_loader, _ = get_loaders(
        CONFIG['data_dir'], CONFIG['batch_size'])

    model = AstroClassifier(CONFIG['num_classes'],
                            CONFIG['backbone']).to(DEVICE)
    criterion = nn.CrossEntropyLoss(label_smoothing=0.1)
    optimizer = optim.AdamW(model.parameters(),
                             lr=CONFIG['lr'],
                             weight_decay=CONFIG['weight_decay'])
    scheduler = CosineAnnealingLR(optimizer,
                                   T_max=CONFIG['num_epochs'])
    scaler = torch.cuda.amp.GradScaler()

    best_val_acc, patience_ctr = 0.0, 0

    for epoch in range(1, CONFIG['num_epochs'] + 1):
        train_loss, train_acc = train_one_epoch(
            model, train_loader, optimizer, criterion, scaler)
        val_loss, val_acc = validate(model, val_loader, criterion)
        scheduler.step()

        print(f"Epoch {epoch:02d}/{CONFIG['num_epochs']} | "
              f"Train Loss: {train_loss:.4f} Acc: {train_acc:.4f} | "
              f"Val Loss: {val_loss:.4f} Acc: {val_acc:.4f}")

        # Save best model
        if val_acc > best_val_acc:
            best_val_acc = val_acc
            patience_ctr = 0
            torch.save({'epoch': epoch,
                        'model_state': model.state_dict(),
                        'optimizer_state': optimizer.state_dict(),
                        'best_val_acc': best_val_acc,
                        f'{CONFIG['checkpoint_dir']}/best_model.pth'})
            print(f" ■ Saved best model (val_acc={val_acc:.4f})")
        else:
            patience_ctr += 1
            if patience_ctr >= CONFIG['patience']:
                print("Early stopping triggered!")
                break

    print(f"\nTraining complete. Best Val Accuracy: {best_val_acc:.4f}")

if __name__ == '__main__':
    main()

```

8. Evaluation & Metrics

evaluate.py

```
import torch
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.metrics import (classification_report, confusion_matrix,
                             accuracy_score, f1_score)
from dataset import get_loaders, CLASSES
from model import AstroClassifier

DEVICE = torch.device('cuda' if torch.cuda.is_available() else 'cpu')

def load_model(checkpoint_path, num_classes=5, backbone='efficientnet_b3'):
    model = AstroClassifier(num_classes, backbone)
    ckpt = torch.load(checkpoint_path, map_location=DEVICE)
    model.load_state_dict(ckpt['model_state'])
    model.to(DEVICE)
    model.eval()
    return model

def evaluate(model, test_loader):
    all_preds, all_labels = [], []
    with torch.no_grad():
        for images, labels in test_loader:
            images = images.to(DEVICE)
            outputs = model(images)
            preds = outputs.argmax(dim=1).cpu().numpy()
            all_preds.extend(preds)
            all_labels.extend(labels.numpy())

    acc = accuracy_score(all_labels, all_preds)
    f1 = f1_score(all_labels, all_preds, average='macro')
    cm = confusion_matrix(all_labels, all_preds)
    report = classification_report(all_labels, all_preds,
                                  target_names=CLASSES)

    print(f"Test Accuracy : {acc:.4f}")
    print(f"Macro F1 Score: {f1:.4f}")
    print("\nClassification Report:")
    print(report)

    # Plot confusion matrix
    plt.figure(figsize=(8, 6))
    sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
                xticklabels=CLASSES, yticklabels=CLASSES)
    plt.title('Confusion Matrix')
    plt.ylabel('True Label')
    plt.xlabel('Predicted Label')
    plt.tight_layout()
    plt.savefig('results/confusion_matrix.png', dpi=150)
    plt.show()
    return acc, f1

if __name__ == '__main__':
    _, _, test_loader = get_loaders('data/processed', batch_size=32)
    model = load_model('checkpoints/best_model.pth')
```

```
evaluate(model, test_loader)
```

9. Grad-CAM Explainability

Grad-CAM generates heatmaps that highlight which image regions drove the model's prediction. This fulfills the **Interpretability** and **Explainability & Visualization** requirements (15% weightage).

gradcam.py

```
import torch
import numpy as np
import cv2
from PIL import Image
from pytorch_grad_cam import GradCAM
from pytorch_grad_cam.utils.model_targets import ClassifierOutputTarget
from pytorch_grad_cam.utils.image import show_cam_on_image
import albumentations as A
from albumentations.pytorch import ToTensorV2
from model import AstroClassifier

DEVICE = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
CLASSES = ["spiral_galaxy", "elliptical_galaxy",
            "nebula", "star_cluster", "planetary"]

def preprocess_image(img_path):
    transform = A.Compose([
        A.Resize(224, 224),
        A.Normalize(mean=[0.485, 0.456, 0.406],
                     std=[0.229, 0.224, 0.225]),
        ToTensorV2()
    ])
    img = np.array(Image.open(img_path).convert('RGB'))
    original = cv2.resize(img, (224, 224)) / 255.0
    tensor = transform(image=img)['image'].unsqueeze(0)
    return tensor.to(DEVICE), original.astype(np.float32)

def run_gradcam(img_path, checkpoint_path, save_path='results/gradcam.png'):
    # Load model
    model = AstroClassifier(num_classes=5, backbone='efficientnet_b3')
    ckpt = torch.load(checkpoint_path, map_location=DEVICE)
    model.load_state_dict(ckpt['model_state'])
    model.to(DEVICE).eval()

    # Target layer (last conv block of EfficientNet)
    target_layer = [model.backbone.blocks[-1]]

    input_tensor, rgb_img = preprocess_image(img_path)

    with GradCAM(model=model, target_layers=target_layer) as cam:
        grayscale_cam = cam(input_tensor=input_tensor,
                           targets=None)[0]
        visualization = show_cam_on_image(rgb_img,
                                         grayscale_cam,
                                         use_rgb=True)

    # Get prediction
    with torch.no_grad():
        output = model(input_tensor)
        pred_idx = output.argmax(1).item()
```

```
    pred_class = CLASSES[pred_idx]
    confidence = torch.softmax(output, 1)[0, pred_idx].item()

    # Save & return
    cv2.imwrite(save_path, cv2.cvtColor(visualization, cv2.COLOR_RGB2BGR))
    print(f"Prediction: {pred_class} ({confidence:.1%})")
    print(f"Grad-CAM saved to {save_path}")
    return pred_class, confidence, visualization

if __name__ == '__main__':
    run_gradcam('sample_image.jpg', 'checkpoints/best_model.pth')
```


10. Streamlit Web App

app/app.py

```
import streamlit as st
import torch
import numpy as np
from PIL import Image
import sys; sys.path.append('.')
from src.gradcam import run_gradcam
from src.model import AstroClassifier
import albumentations as A
from albumentations.pytorch import ToTensorV2
import tempfile, os

CLASSES = ["Spiral Galaxy", "Elliptical Galaxy",
            "Nebula", "Star Cluster", "Planetary Object"]
DEVICE = torch.device('cuda' if torch.cuda.is_available() else 'cpu')

@st.cache_resource
def load_model():
    model = AstroClassifier(num_classes=5, backbone='efficientnet_b3')
    ckpt = torch.load('checkpoints/best_model.pth', map_location=DEVICE)
    model.load_state_dict(ckpt['model_state'])
    model.to(DEVICE).eval()
    return model

st.set_page_config(page_title="SCALE x ODYSSEY", page_icon="■", layout="wide")
st.title("■ SCALE x ODYSSEY")
st.subheader("Astronomical Object Classifier")

uploaded = st.file_uploader("Upload an astronomical image",
                             type=['jpg', 'png', 'jpeg'])

if uploaded:
    col1, col2 = st.columns(2)
    with col1:
        st.image(uploaded, caption="Uploaded Image", use_column_width=True)

        # Save temp file
        with tempfile.NamedTemporaryFile(suffix='.jpg', delete=False) as tmp:
            tmp.write(uploaded.read())
            tmp_path = tmp.name

        with st.spinner("Classifying..."):
            model = load_model()
            pred_class, confidence, gradcam_img = run_gradcam(
                tmp_path, 'checkpoints/best_model.pth')

    with col2:
        st.image(gradcam_img, caption="Grad-CAM Explanation",
                 use_column_width=True)
        st.success(f"""{CLASSES[CLASSES.index(pred_class.replace('_', ' ').title())]}""")
        st.metric("Confidence", f"{confidence:.1%}")

    os.unlink(tmp_path)
```

Run the App

```
# From the project root  
streamlit run app/app.py
```

```
# The app will open at http://localhost:8501
```

11. Evaluation Criteria Checklist

Component	Weight	How to Achieve
Classification Performance	40%	>80% accuracy with EfficientNet + augmentation
Model Efficiency	15%	Use B3 (12M params), mixed precision, batch inference
Explainability & Visualization	15%	Grad-CAM heatmaps + attention visualization
Innovation / Bonus Features	15%	Image captioning, anomaly detection, web app
Documentation & Presentation	15%	Clean README, eval report, demo video

Bonus Tasks — Quick Wins

Bonus 1: Image Captioning

```
# Use BLIP from HuggingFace
from transformers import BlipProcessor, BlipForConditionalGeneration
from PIL import Image

processor = BlipProcessor.from_pretrained("Salesforce/blip-image-captioning-base")
model_cap = BlipForConditionalGeneration.from_pretrained(
    "Salesforce/blip-image-captioning-base")

def caption_image(img_path):
    image = Image.open(img_path).convert('RGB')
    inputs = processor(image, return_tensors="pt")
    out = model_cap.generate(**inputs, max_new_tokens=50)
    return processor.decode(out[0], skip_special_tokens=True)

print(caption_image("nebula.jpg"))
# "a colorful nebula with bright stars and gas clouds"
```

Bonus 2: Anomaly Detection

```
# Use softmax confidence threshold
def detect_anomaly(logits, threshold=0.5):
    probs = torch.softmax(logits, dim=1)
    max_conf = probs.max(dim=1).values
    is_anomaly = max_conf < threshold
    return is_anomaly, max_conf

# If max confidence < 50%, flag as unknown/anomaly
```

12. Submission Checklist

- [] Trained model saved as best_model.pth with >80% test accuracy
- [] Source code in clean GitHub repository
- [] requirements.txt with pinned versions
- [] README.md with setup and inference instructions
- [] Evaluation report (confusion matrix, per-class metrics)
- [] Grad-CAM visualizations for each class
- [] Streamlit/Flask demo running (hosted or local with instructions)
- [] Final presentation / demo video (1–2 min)
- [] All experiments documented (architectures tried, hyperparams)
- [] Reproducible training script (random seed fixed)

■ ■ Fix your random seed for reproducibility: `torch.manual_seed(42)`, `np.random.seed(42)`

■ Attribution: Use pretrained models from TIMM/HuggingFace with proper citations in README. Commercial black-box APIs (OpenAI Vision etc.) should NOT be the primary solution.

Good luck! ■ — SCALE × ODYSSEY